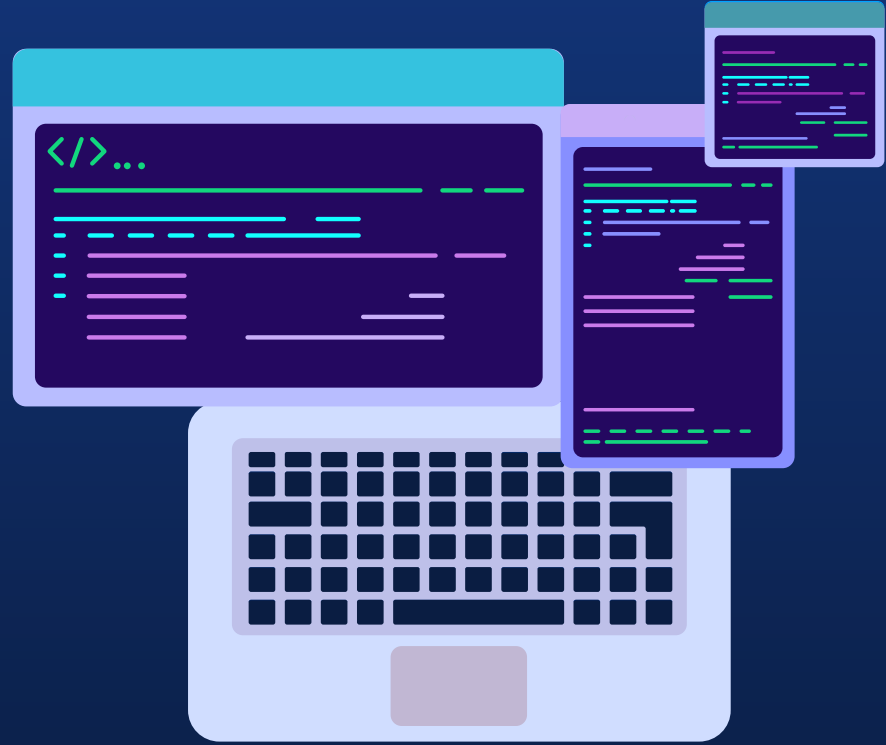


Java O-O Program Visualizer/Animator

By Prottasha Deb, Angelina Martis,
Arya Patel & Aiden Stensrud





Background

- Novice programmers in classes such as CSE 1001 and CSE 1002 often struggle to grasp the **concept** of classes & objects, particularly in **Java programs**.



VISIBILITY

The inner workings of the program are “invisible” to the programmer



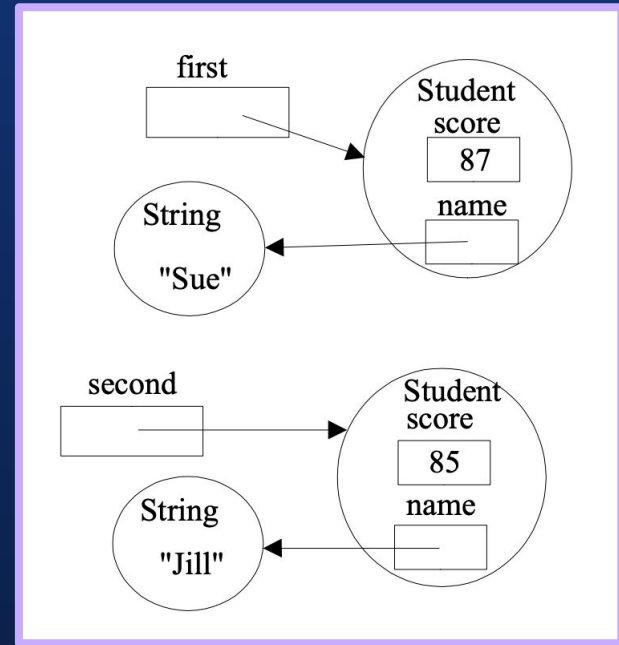
SPEED

The program’s instructions execute in rapid succession



Solution: Memory Diagrams

- Memory diagrams provide a **high-level representation** of the internal memory of an object-oriented program at a given state
- ex: The following memory diagram displays two **Student** objects within a program's memory





Our goal is to design a
program that **animates** a
memory diagram which
mimics the execution of a
given Java program



Approach - Key Features



USER INPUT

Users can input a Java program into the visualizer, which is parsed and converted into instructions for the visualizer regardless of user syntax.



VISUALIZE

The visualizer will then display the input program's relevant memory data as a memory diagram, displaying classes, objects, and references that have been created or assigned.



ANIMATE

Users can iterate through the program line-by-line and see the memory diagram shift and change as it executes instructions at the user's own pace

Example

```
int myNumber = 1;  
myNumber = 2;  
int myOtherNumber = 3;
```

myNumber

1

Example

```
int myNumber = 1;  
myNumber = 2;  
int myOtherNumber = 3;
```

myNumber

2

Example

```
int myNumber = 1;  
myNumber = 2;  
int myOtherNumber = 3;
```

myNumber

2

myOtherNumber

3



Novel Functionalities

- A Java O-O Visualizer was previously attempted by a Senior Design group & they could **not** successfully focus on **accurate memory diagrams!**
- We plan to overcome this obstacle with a **novel approach**: implementing common compiler strategies to parse the Java programs **before** they become memory diagram instructions.
- This will allow us to:
 - Place memory visualization at the center of the system.
 - Focus on **accurately** representing Java program execution within memory diagrams.



Algorithms & Tools

MEMORY DIAGRAMS

Represent the program's memory in a visual format



COMPILER TECHNIQUES

Lexical analyzer, syntax analyzer, etc. to process the Java program



ANIMATION ENVIRONMENT

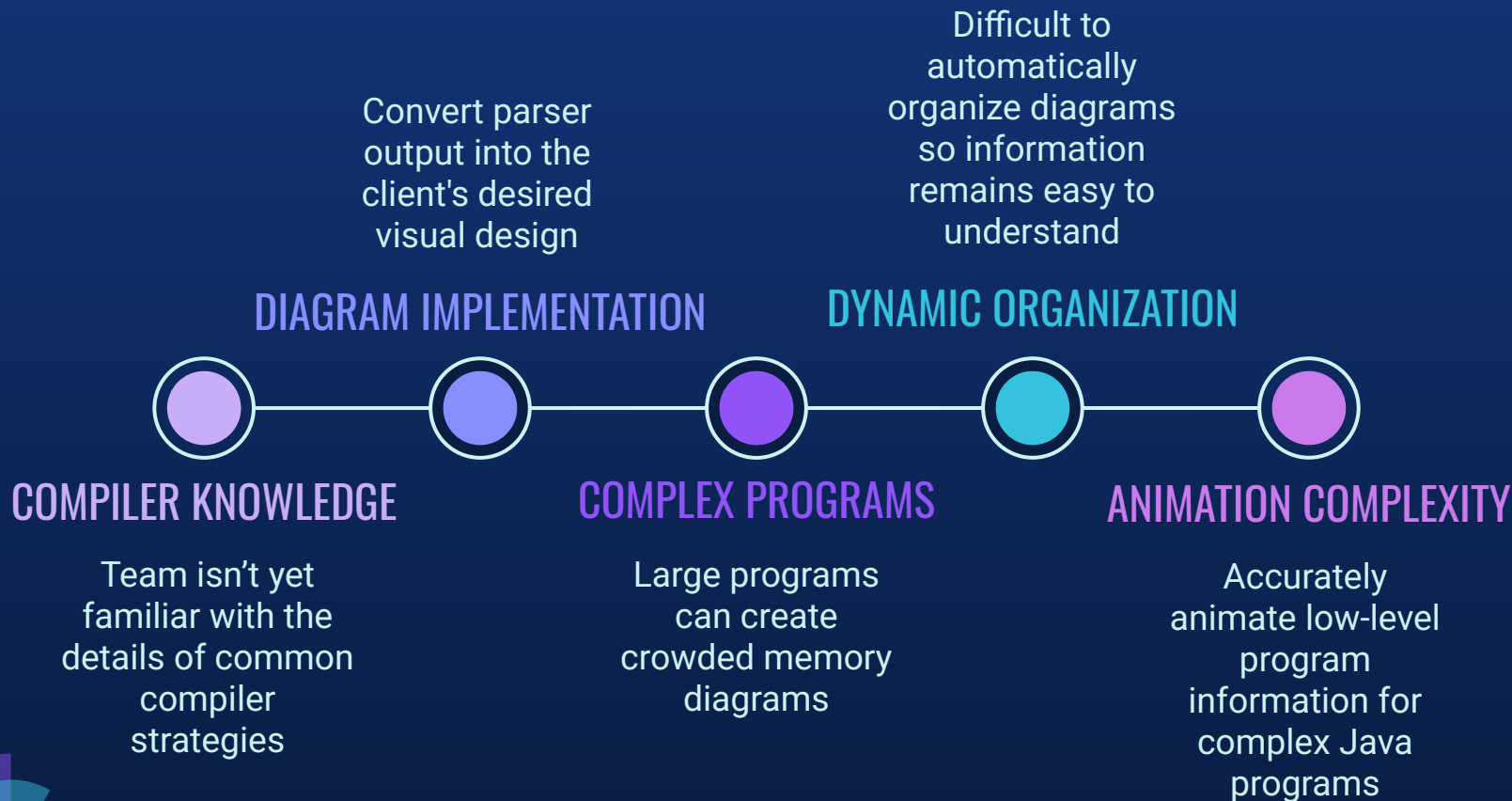
p5.js, Matplotlib, etc. to facilitate the memory diagram animations



CODE INTERPRETER

Interpret the memory diagram instructions line-by-line to animate each machine state

Technical Challenges



Milestone 1 (next milestone)

- **Compare & Research:**
 - Collaboration Tools for Software Development, docs & presentations, communication, task calendar, etc
 - Optimal application to implement the visualizer (web or desktop?)
 - Tools & strategies for Java code parsing
- **Resolve** Technical Challenge: Lack of compiler knowledge
- **Design & Demo** first draft for the visualizer output memory diagrams
- **Write** requirement, design, and test plan documentation

Milestone 2 (future milestone)

- **Discover** the optimal tool to facilitate memory diagram display
- **Implement, Test, & Demo:**
 - Java code parser
 - Simple memory diagram generator
- **Conceptualize & Draft** memory diagram animations
- **Resolve** Technical Challenge: Memory diagram design for complicated memory states

Milestone 3 (future milestone)

- **Implement, Test, & Demo:**
 - Memory Diagram Animator
 - Memory Diagram Generator for Complex Memory States
 - Full Cohesive Application that combines the Parser, Generator, and Animator
 - UI for chosen type of Application
- **Explore** possible extensions (if applicable)

Task Matrix for Next Milestone



Task	Arya	Aiden	Prottasha	Angelina
Resolve technical challenges	-	lack of compiler knowledge	-	-
Design memory diagram demo	design 50%	give input	give input	design 50%
Compare and select the application type	web	-	-	desktop
Compare and select Collaboration Tools	presentation	programs	documents	communication, task calendar
Compare and select technical tools/strategies	-	java code parser	memory instruction animator	-
Requirement Document	write 10%	write 30%	write 50%	write 10%
Design Document	write 10%	write 50%	write 30%	write 10%
Test Plan	write 10%	write 30%	write 50%	write 10%





THANK YOU FOR
LISTENING!

Any Questions?

